

An Analysis of Flow-based generative model

Flow-based generative model

Flow-based generative model -- Part 1

Let z_0 be a (possibly multivariate) random variable with distribution $p_0(z_0)$. For $i = 1, \dots, K$, let $z_i = f_i(z_{i-1})$ be a sequence of random variables transformed from z_0 . The functions $f_1, \dots, f_K$ should be invertible, i.e. the inverse function f_i^{-1} exists. The final output z_K models the target distribution. The log likelihood of z_K is (see derivation):

Flow-based generative model -- Part 2

Non-zero vectors in $\mathbb{R}^n$ are projected onto the unit sphere as: $\pi(z) = (\sum_{i=1}^n z_i^2)^{-1/2} z$

Flow-based generative model -- Part 3

In fact, the set of f_{gcal} transformations form a group under function composition. The set of f_{cal} transformations form a subgroup. Also see: Dirichlet calibration, which generalizes f_{gcal} , by not placing any restriction on the matrix, A , so that invertibility is not guaranteed. While Dirichlet calibration is trained as a discriminative model, f_{gcal} can also be trained as part of a generative calibration model.

Flow-based generative model -- Part 4

$\arg \max_{\theta} \sum_{i=0}^N \log p_{\theta}(x_i)$

Flow-based generative model -- Part 5

Derivation of log likelihood Consider z_1 and z_0 . Note that $z_0 = f_1^{-1}(z_1)$. By the change of variable formula, the distribution of z_1 is:

Flow-based generative model -- Part 6

Learning probability distributions by differentiating such log Jacobians originated in the Infomax (maximum likelihood) approach to ICA, which forms a single-layer ($K=1$) flow-based model. Relatedly, the single layer precursor of conditional

generative flows appeared in. To efficiently compute the log likelihood, the functions $f_1, \dots, f_K$ should be easily invertible, and the determinants of their Jacobians should be simple to compute. In practice, the functions $f_1, \dots, f_K$ are modeled using deep neural networks, and are trained to minimize the negative log-likelihood of data samples from the target distribution. These architectures are usually designed such that only the forward pass of the neural network is required in both the inverse and the Jacobian determinant calculations. Examples of such architectures include NICE, RealNVP, and Glow.

Flow-based generative model -- Part 7

To derive the inverse transform, with suitable restrictions on the parameters to ensure invertibility. To derive in simple closed form the differential volume ratio, R_f . An interesting property of these simple spherical flows is that they don't make use of any non-linearities apart from the radial projection. Even the simplest of them, the normalized translation flow, can be chained to form perhaps surprisingly flexible distributions.

Flow-based generative model -- Part 8

Differential volume ratio for curved manifolds Consider a flow, $y = f(x)$ on a curved manifold, for example S^{n-1} which we equip with the embedding function, e that maps a set of $(n-1)$ angular spherical coordinates to S^{n-1} . The Jacobian of e is non-constant and we have to evaluate it at both input (E_x) and output (E_y) . The same applies to r , the representation function that recovers spherical coordinates from points on S^{n-1} , for which we need the Jacobian at the output (R_y) . The differential volume ratio now generalizes to:

Flow-based generative model -- Part 9

$x'x = 1 \implies y = f_{\text{aff}}(x; M, c) = f_{\text{lin}}(x; M + cx')$

Flow-based generative model -- Part 10

which maps a conveniently chosen, $(n-1)$ -dimensional representation, $p \sim$

An Analysis of Flow-based generative model

Flow-based generative model

$\{p\} \}$, to the embedded manifold. The n -by- $(n-1)$ Jacobian is

Flow-based generative model -- Part 11

so that the factor $|\det \mathbf{D}|$ cancels in the volume ratio, which can now already be numerically evaluated. It can however be rewritten in a sometimes more convenient form by also introducing the representation function, $r: p \sim \tilde{p}$, which simply extracts the first $(n-1)$ components. The Jacobian is $\mathbf{R} = \begin{bmatrix} \mathbf{I} & \mathbf{0} \end{bmatrix}$. Observe that, since $e \circ r \circ f = f$, the chain rule for function composition gives: $\mathbf{E} \mathbf{R} \mathbf{F} p = \mathbf{F} p$. By plugging this expansion into the above Gram determinant and then refactoring it as a product of determinants of square matrices, we can extract the factor $|\det (\mathbf{E}' \mathbf{E})| = n$, which now also cancels in the ratio, which finally simplifies to the determinant of the Jacobian of the "sandwiched" flow transformation, $r \circ f \circ e$:

Flow-based generative model -- Part 12

$R_{\text{trans}}(x; c) = \frac{1 + x' c \mathbf{x} + c \mathbf{n}}{\sqrt{\mathbf{x} \mathbf{x} + \mathbf{c} \mathbf{c}}}$